

Asynchronous Federated Split Learning

R.A. Albuquerque, L.P. Dias and Junior Momo Ziazet
Computer Science and Software Engineering
Concordia University
Montreal (Qc), Canada

K. Vandikas and S. Ickin
ER/AI
Ericsson
Kista, Sweden

B. Jaumard
GAIA
Ericsson
Montreal (Qc), Canada

C. Natalino, L. Wosinska, P. Monti
Optical Networks
Chalmers University of Technology
Gothenburg, Sweden

E. Wong
The University of Melbourne
Electrical and Electronic Engineering
Melbourne, Australia

Abstract—We propose a first Asynchronous Federated Split Learning (AFSL), to add the flexibility of asynchronous computing to the combination of federated and split learning. This amounts to designing a harmonious combination of different paradigms in order to benefit from the advantages of each of them and to reduce the impacts of their shortcomings.

This way, AFSL answers to the increasingly rising interest for distributed algorithms with the advent of edge computing in order to support new market segments, such as cloud gaming, immersive eXtended Reality (XR), indoor positioning, and mission critical IoT networks, with stringent requirements on latency and reliability.

Computational experiments are conducted on IID and non-IID datasets to investigate the added value of the asynchronous feature. Results indicate that AFSL can accelerate model learning by up to 86% without sacrificing the model's convergence and accuracy. Indeed, not only average training times are reduced, but clients use fewer resources, a critical characteristic for devices with limited computing capabilities, e.g., in edge devices. Performance degradation can be mitigated by a careful selection of the aggregation principle. Other advantages are with AFSL training in dynamic scenarios as it provides robustness with a short recovery time by leveraging asynchronous client training.

Index Terms—Distributed machine learning, Federated learning, Split learning, Asynchronous, Edge computing

I. INTRODUCTION

The landscape of wireless communication has seen remarkable advancements with the introduction of fifth-generation (5G) and the upcoming sixth-generation (6G) networks. Unparalleled connectivity, ultra-low latency, and staggering data rates are expected in 5G, enabling massive internet of things (IoT) applications, network function virtualization (NFV) and machine learning (ML) methods for network optimization [2]. In fact, to improve performance, efficiency and customer experience, artificial intelligence (AI) and ML can be seen as crucial tools for the telecom industry in 5G networks and beyond [3]. Currently, in 5G networks, AI is generally being incorporated in network components by: 1) replacing existing rule-based algorithms, 2) adding a new AI-based components, 3) adding AI-based control to the existing legacy components [3]. In the future, in 6G

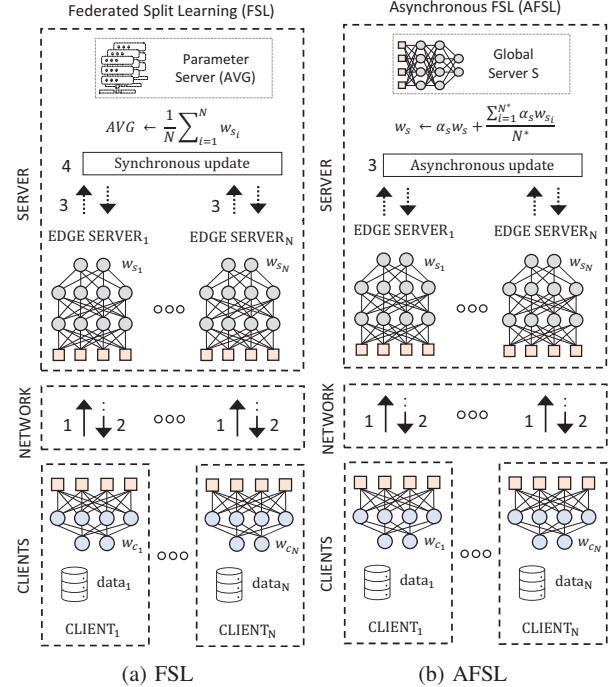


Fig. 1: FSL [1] and AFSL representation. The model is split into client and server sub-models. Contrary to FSL, in AFSL the edge servers' weights are updated in an asynchronous way.

networks, AI native network components are expected. In this context, heterogeneous devices with different compute capabilities, generating a high amount of highly dispersed and heterogeneous data, bring challenges to this new era.

On top of that, traditional ML training methods rely on data centralization, usually located far-away from where data is originally collected. However, with the growth in size and sensitivity concerns of training data, centralized ML solutions may not meet the low latency and privacy requirements of

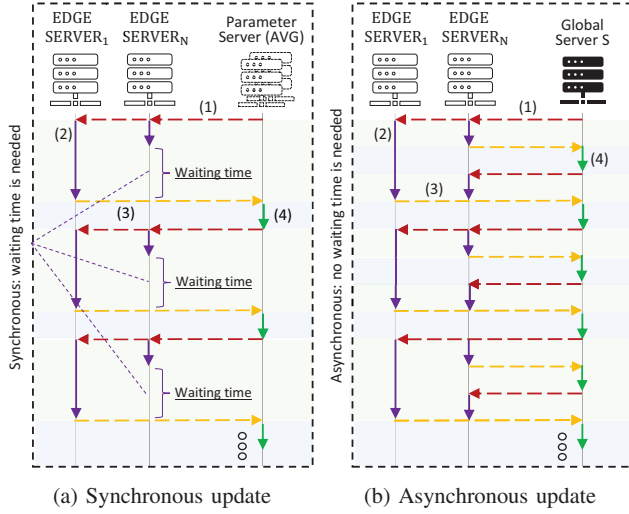


Fig. 2: Sequence diagram of synchronous and asynchronous updates. After the client has completed its epoch in synchronous update, it has to wait for all other clients before aggregation, whereas in asynchronous update, it doesn't wait.

modern systems and society [4]. To alleviate these challenges, distributed ML techniques have been proposed.

In a Federated Learning (FL) algorithm [5], each user with its own local dataset, trains one entire deep learning model locally and each model has the same architecture. At the end of each training round, the trained models (weights) from all clients are aggregated via arithmetic averaging, i.e., FedAvg, in a server. Then, a copy of this new aggregated model, known as global model, is sent back to the clients. This process occurs iteratively during the training phase. One advantage of FL is the framework's scalability as multiple models are trained in parallel during each epoch [1]. However, as training an entire model is often computationally demanding, FL may not be suitable for resource-constrained devices, e.g., Internet-of-Things [4]. Moreover, although data remains in the client during the entire training, the weights are shared and can be used to reconstruct the source dataset, e.g. inversion attacks, leading to a potential data leakage [1].

To partially overcome these limitations, split learning (SL) has been proposed [6]. In a SL algorithm, a deep neural network is split into at least two parts namely the client-side and server-side. On the client-side, the first part of the model which is usually composed of a few layers, uses its local data and sends the intermediate data, i.e., output of the client-side model called smashed data, to the server-side. At the server-side, the smashed data is used as input to complete the forward propagation. During the back-propagation, the gradients are computed at the server and sent back to the client-side, completing one client epoch. Then, sequentially, the next client starts the training process on its data partition, following the same steps previously described. The advantage of SL compared to FL is that each client has only to implement

a small fraction of the neural network model, reducing the computational requirements on the client side [4]. More importantly, in SL, the server has no access to clients' weights and data, thus preserving the clients' data privacy. However, since the training process is sequential, the vanilla SL [6] does not consider computational heterogeneity, leading to computation resource under-utilization. In addition, as presented in [1], the model may not converge if the dataset in each client has unbalanced features, i.e., non-independent and identically distributed data (non-IID).

More recently, to take advantage of the FL scalability, SL reduced computational requirement for clients and SL's stronger data privacy, combinations of FL and SL have been proposed in the literature. In [7], an algorithm called Parallel Split Learning (PSL) is proposed. Unlike the vanilla sequential SL, in PSL, the training phase on the client-side is parallelized. However, as the system shares a single server-side model, the backward propagation can only start when all clients have finished the forward phase. Therefore, this may have a great impact on the training time if the clients' devices have different computation capabilities. In [8] a novel approach called SplitFed Learning (SFL) combines the advantages of FL and SL. In contrast to PSL, multiple instances of the server-side model are deployed and shared across the clients, letting them perform the forward propagation in parallel and independently. At the end of each epoch, all client-side and server-side models are aggregated and updated. In [9], Gao *et al.*, evaluate FL, SL, and SFL performance under a real resource-constrained device environment. In addition, the authors proposed a so-called generalized SFL (SFLG), which provides a simplified aggregation scheme (based on grouping) that is suitable for server-side with limited computational capacity. However, although improvements in terms of training convergence time were achieved, the SFLG weights aggregation is still synchronous and does not explore the full potential of an asynchronous FL-SL combination.

Similarly to [8] and [9], Turina *et al.* [1] propose the FSL framework. Therein, each client has its own copy of the server-side, but contrary to SFL and SFLG, it only considers model synchronization for the server-side. Since FSL does not share clients' weights, it is more resilient in terms of privacy. However, as the server-side weights are aggregated in a synchronous way (as in SFL and SFLG) at the end of each epoch, FSL may also suffer in scenarios with heterogeneous devices. In other words, faster clients with higher computational resources and/or higher network throughput may be idle during the training phase (leading to resource under-utilization and longer training time). Fig. 1a depicts the FSL architecture and its operation. Each client has one exclusive edge server and exchanges intermediate data and gradients over the network (1,2). After the last batch of data (end of an epoch), all the clients send the edge server's weights to a parameter server (3). The parameter server averages all the edge servers' weights only when all clients have finished their epoch (4).

Despite the ongoing research efforts towards FL-SL approaches, significant work remains to be done regarding

heterogeneous clients with non-IID data, a common characteristic of many FL-SL applications, such as IoT devices with varying types and sizes of data. Several asynchronous frameworks have been proposed for FL [10] and SL. For instance, in [11], the authors introduced WKAF, a two-stage weighted K asynchronous FL framework, which addresses non-IID data challenges through adaptive learning rates and consistent gradient selection. In [12], authors introduced the Asynchronous Online FL (ASO-Fed) framework to handle heterogeneous edge devices by asynchronously updating the central model. To minimize communication overhead in split learning, an asynchronous training scheme is introduced in [13], selectively updating the client-side model. Additionally, in [14], authors proposed an asynchronous scheme by profiling the participants. Most asynchronous FL literature focuses on addressing the straggler problem, while asynchronous SL literature concentrates on reducing communication overhead. While various asynchronous frameworks have already been proposed for FL and SL, to the best of our knowledge, only one paper [15] has introduced an asynchronous framework combining FL and SL. Nonetheless, it still suffers from suboptimal resource utilization and longer training time. Specifically, participants must wait until the medoid they belong to is selected before being able to participate in the aggregation. Additionally, aggregation only occurs when all participants of the selected medoid have completed their client-side epochs.

Given these challenges, we propose a novel approach termed asynchronous federated split learning (AFSL), which aims to work asynchronously amongst the clients while taking care of their heterogeneity without compromising learning accuracy and convergence time. The proposed asynchronous scheme provides the means of complementing FL-SL combination approaches. Our work differs from [11]–[14] by addressing a combined setting of split and federated learning. Unlike [14], we eliminate the need for participant profiling overhead. Furthermore, in contrast to [13], our work accounts for the presence of heterogeneous devices with varying capabilities. In the scope of this work we exclude the impact of secure aggregation [16].

The paper is organized as follows. Section II introduces the asynchronous scheme, beginning with the asynchronous algorithm and detailing the associated training procedure. Section III presents comprehensive computational experiments, comparing AFSL with synchronous FSL and evaluating its performance across diverse datasets, including both independent and identically distributed (IID) and non-IID datasets. Finally, Section IV concludes the paper.

II. ASYNCHRONOUS FEDERATED SPLIT LEARNING

This section outlines the proposed asynchronous federated split learning scheme, detailing the system model, asynchronous update principle, and training procedure.

A. System model

In a given communication network, we consider a scenario in which N heterogeneous devices collaborate with an edge

server to train a deep neural network model in a distributed manner. Each device i has its own local data set D_i , and the data sets are distributed in a non-IID fashion across devices. As in a real-world scenario, devices may have relatively limited and different computational capabilities, with some devices being more powerful than others. The architecture of the deep neural network is composed of several layers which are partitioned in two parts. As shown in Fig. 1, each device executes the first partition, usually composed of a few layers with weights W_{c_i} . The second partition, composed of more layers with weights W_s , runs on a remote server (for example at the edge of the network). At the server side, each device i is trained with a server-side copy W_{s_i} of the global server weights W_s . At the end of each epoch, the trained copy W_{s_i} is used to update the global server weights W_s in an asynchronous way, as described in the next section.

B. Asynchronous updating principle

With AFSL, we introduce a parallel learning protocol to boost the learning speed of federated split learning approaches. Indeed, with synchronous updating, when a client i completes its epoch, it must wait until all other clients participating in the training to complete their corresponding epoch before it can receive the aggregated weights W_s and move on to the next epoch. Many real-world environments are dynamic and heterogeneous. Devices may have different computational capabilities and network-varying conditions (e.g., latency, bit rate). In such a scenario, the slowest device, i.e. the one that ends its epoch last, will determine the training duration until convergence. This will lead to resource under-utilization and longer training times. This problem can be alleviated by introducing asynchronous updating. Fig. 1b illustrates the AFSL architecture and its operation. Similar to FSL, each client has one exclusive edge server and exchanges intermediate data and gradients over the network (1,2). However, contrary to FSL, in AFSL the edge servers' weights are updated independently, in an asynchronous way (3). In other words, after the last batch of data (end of an epoch) of a given client, the client may be ready to perform aggregation, depending on the adopted strategy. To better understand this new mechanism, Fig. 2b shows a sequence diagram of the asynchronous update. After receiving the global server weights (1), the client completes its epoch (2), and can directly submit its trained weights (3) to aggregation (4). Then, the client moves on to the next epoch without being affected (put into inactive mode) by the training speed of other devices, as is the case of synchronous updating shown in Fig. 2a. As a result, in asynchronous update, faster clients terminate their total epoch earlier, thus consuming fewer resources.

1) *Remark 1. Asynchronous aggregation principles:* It is important to note that in AFSL, asynchronous aggregation can be performed using various strategies tailored to specific use cases or desired design preferences. Therefore, asynchronous updates can be implemented based on different principles, which makes them highly adaptable to various device heterogeneity situations and advantageous in various

scenarios. Model performance can then be further improved using different aggregation principles, including

- directly when a model ends its epoch or,
- after receiving a given percentage of models
- after a certain time threshold.

Therefore, at any point during the training process, the global server weights W_s can be updated with the number of models ready for aggregation (N^*), utilizing an aggregation equation such as:

$$W_s \leftarrow \alpha_s W_s + \frac{\sum_{i=1}^{N^*} \alpha_{s_i} W_{s_i}}{N^*}, \quad (1)$$

where $N^* \leq N$ is the number of clients ready for aggregation at a given time based on the chosen asynchronous aggregation principle. α_s and α_{s_i} are, respectively, the component-specific weights regarding the global server and the edge server of each client i . These two parameters are used to control the update weighting. They are defined so that $\alpha_s + \sum_{i=1}^{N^*} \alpha_{s_i} = 1$, i.e., these parameters are not set as a unique constant but are varying based on N^* (e.g., considering a case where α_s and all the α_{s_i} are the same, we have $\alpha_s = \alpha_{s_i} = \frac{1}{N^*+1}$).

2) *Remark 2. Dynamic update of α_s and α_{s_i} parameters:*

As previously mentioned, the number of models ready for aggregation N^* is directly influenced by the chosen aggregation principle, leading to different ranges of values for N^* . This introduces instability in weight aggregation if the weighting parameters, α_s and α_{s_i} , are not carefully selected. For instance, when opting for the design choice where $\alpha_s = \alpha_{s_i} = \frac{1}{N^*+1}$, i.e., assigning equal attention to both parameter server weights and client-ready model weights, there is a risk of instability, particularly when W_{s_i} distributions significantly differs from W_s distribution and N^* is small (e.g., $N^* \leq 2$). Given the potential for instability, ensuring a stable model and preventing abrupt changes in the parameter server W_s , which could adversely impact the training of other clients, underscores the critical importance of carefully choosing the parameters α and α_{s_i} .

It is noticeable that a great number of combinations can be done with α_s and α_{s_i} parameters. To attain performance comparable to synchronous approaches (e.g., FSL) while minimizing convergence time, a key attribute of our proposed AFSL lies in how we aggregate the ready models with the global parameter server. Indeed, we proposed implementing a dynamic parameter updates, wherein α_s and α_{s_i} can vary based on the state (i.e., maturity) of the parameter server model at each aggregation step. This variation could be tied to factors such as the client epoch or the server aggregation number. We proposed a formulation where the edge servers' weight α_{s_i} exponentially decays over the number of server aggregations referred to as e_s . In other words, depending on the server's maturity (the higher the aggregation step, the more mature the global server model is), more weight is given to the global server during the aggregation. We propose to define the edge server weights α_{s_i} and the global server weight α_s as follows:

$$\alpha_{s_i} = \frac{\sigma \beta^{e_s}}{N^*}, \quad \alpha_s = 1 - \sum_{i=1}^{N^*} \alpha_{s_i}, \quad (2)$$

Algorithm 1 Asynchronous Federated Split Learning (AFSL)

```

1: procedure CLIENT ▷ running at client side
2:   Initialize client  $i$  weights  $W_{c_i}$  according to a weight
3:   initialization strategy.
4:   for  $epoch$  in  $epochs_i$  do
5:     for  $data_i, labels_i$  in dataset  $D_i$  do
6:        $c\_output_i \leftarrow c\_Forward_i(data_i)$ 
7:       send  $(c\_output_i, labels_i)$  to server
8:        $s\_grad_i \leftarrow$  receive gradient from server
9:        $c\_Backprop_i(s\_grad_i)$ 
10:    end for
11:  end for
12: end procedure
13: procedure SERVER ▷ running at server side
14:   for  $epoch$  in  $epochs_i$  do
15:      $W_{s_i} \leftarrow$  get latest server weights  $W_s$ 
16:     for  $batch$  in  $num\_batch_i$  do
17:        $c\_output_i, labels_i \leftarrow$  receive output and labels
18:       from client  $i$ 
19:        $output_i \leftarrow s\_Forward_i(c\_output_i)$ 
20:        $Loss_i \leftarrow LossFunc(output_i, labels_i)$ 
21:        $grad_i \leftarrow Gradient(Loss_i)$ 
22:        $s\_grad_i \leftarrow s\_Backprop_i(grad_i)$ 
23:       send  $s\_grad_i$  to client  $i$ 
24:     end for
25:     submit weight  $W_{s_i}$  for aggregation to main process
26:   end for
27: end procedure
28: procedure MAIN ▷ running at server side
29:   Initialize global server weight  $W_s$  according to a
30:   weight initialization strategy.
31:   for each connected client  $i$ , in parallel do
32:     Establish training with server by executing
33:     procedures CLIENT () and SERVER ()
34:   end for
35:   while Training ongoing do
36:     get current submitted weights from clients
37:      $W_s \leftarrow$  update server weight (according to Section II-B)
38:   end while
39: end procedure

```

where σ is a parameter that can be manually updated to control the effect of the decay β^{e_s} , with $\beta < 1$ being the decay rate. The parameters σ and β were chosen with a grid search and the selected values were $\sigma = 0.5$ and $\beta = 0.99$.

C. AFSL Training Procedure

As presented in Algorithm 1, AFSL operates as follows. First, the main process initializes the global server weights W_s according to a weight initialization strategy. Then, in parallel, the training of each connected client i with a remote server copy W_{s_i} of W_s is established whereby the weights of the client's local model W_{c_i} are initialized according to the weight initialization strategy. Next, the local client model i performs forward propagation and sends its output c_output_i , i.e., the smashed data, and labels $labels_i$ to the server. The server terminates the forward propagation with the corresponding server copy W_{s_i} and gets the output $output_i$. The server applies a loss function, e.g., sum of squares or cross entropy, to the output $output_i$ to accumulate the errors $Loss_i$. The gradient $grad_i$ of the error is back propagated through the

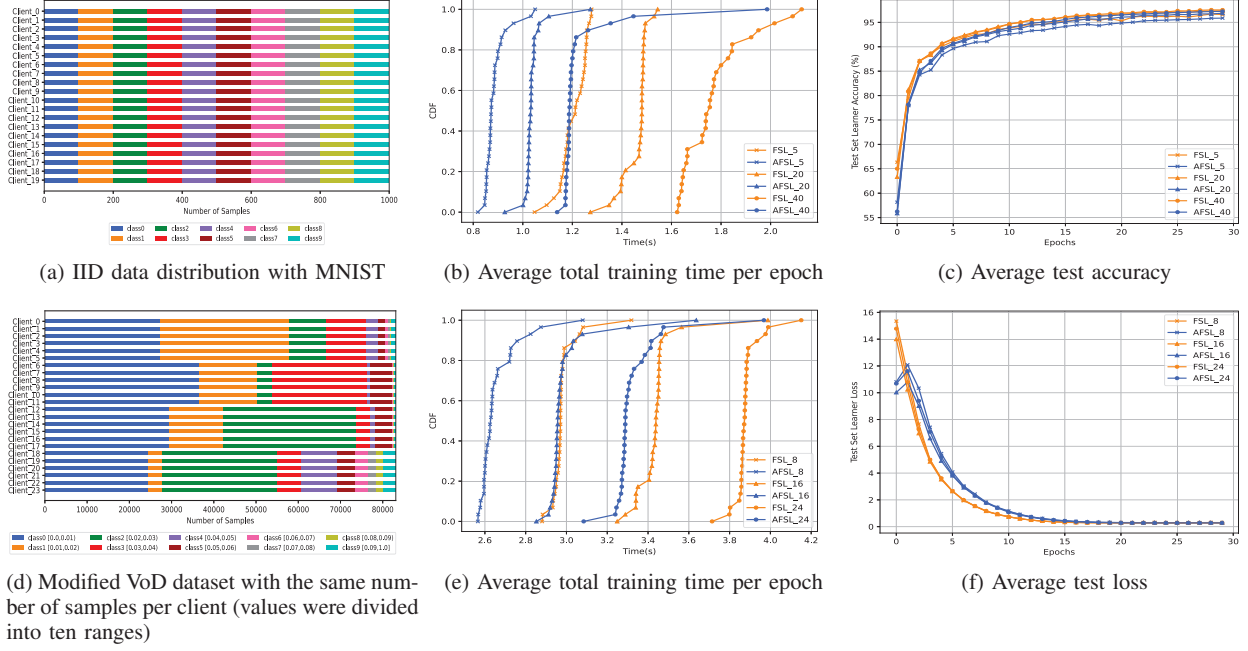


Fig. 3: Experiment I – FSL & AFSL with 5, 20 and 40 clients on MNIST (Figs. 3a, 3b and 3c) and 8, 16, 24 clients on VoD (Figs. 3d, 3e and 3f). In all these experiments, clients have similar computational capabilities with the same number of data samples.

server layers, and the server output gradient s_grad_i is sent to the client, which terminates its back propagation. Once a client completes an epoch, as opposed to a synchronous update involving the aggregation of all models, its trained copy server weights W_{s_i} are used to update the global server model weights W_s according to an asynchronous update principle as defined in subsection II-B. Thus, leading to accelerated training. Eq. (1) shows the general asynchronous aggregation scheme for the global server model, where the parameters α_s and α_{s_i} are dynamically chosen to enhance model stability and performance. The next epoch continues with W_{s_i} updated to the latest W_s , and this is done in parallel for all clients.

III. PERFORMANCE EVALUATION

To quantify the effectiveness of the proposed AFSL, we conducted three experiments, namely Experiments I to III, under different conditions to evaluate training time, model accuracy, and robustness under a dynamic environment. We assume a scenario where a group of N clients train, in a cooperative way, models for a classification and regression problem while keeping their data locally. This context may well suit a wide range of IoT applications, such as home automation, smart security, smart vehicles, speech commands processing, etc. To reproduce such a scenario, for the classification problem, we adopted the well-known MNIST dataset [17] with the LeNet-5 Neural Network [18] and we considered accuracy as the performance metric for this dataset. On the other hand, the evaluation of the regression problem is based on realistic video on demand (VoD) traces [19] obtained from a testbed at KTH

University. Each trace group contains about 1700 features (e.g., CPU utilization per core, memory utilization, and disk I/O) collected from six server machines. Three VoD service-level metrics serve as the target values, i.e., net value of read average delay, average display delay, and average audio play delay. More details about these traces can be found in [20]. We used the normalized mean absolute error as the performance metric to evaluate the models trained with this dataset.

All experiments, were run on a single device with multiple CPUs, emulating a distributed system. The clients and the server were deployed using different terminals, and socket communication was used to guarantee intermediate data transmission between clients and server. All experiments were conducted on a AMD Ryzen Threadripper 2990WX-32-Core processor @ 2.95GHZ with 128 GB of RAM under the Python environment version 3.10.6, with Pytorch 2.0.1 library.

A. Experiment I — FSL vs AFSL with similar clients

For Experiment I, we aim to compare AFSL and FSL under a scenario where the clients have similar computing capabilities and the same number of data samples. To do so, we adopted the MNIST dataset with a balanced IID distribution (as illustrated in Fig. 3a and for the VoD dataset, as the data were non-IID by nature, we ensured the same number of samples per client (as illustrated in Fig. 3d). We also set, empirically, the number of epochs to 30 and conducted three different simulations considering 5, 20, and 40 connected clients on the MNIST dataset and 8, 16, and 24 clients on the VoD dataset. It is worth mentioning that, in this experiment,

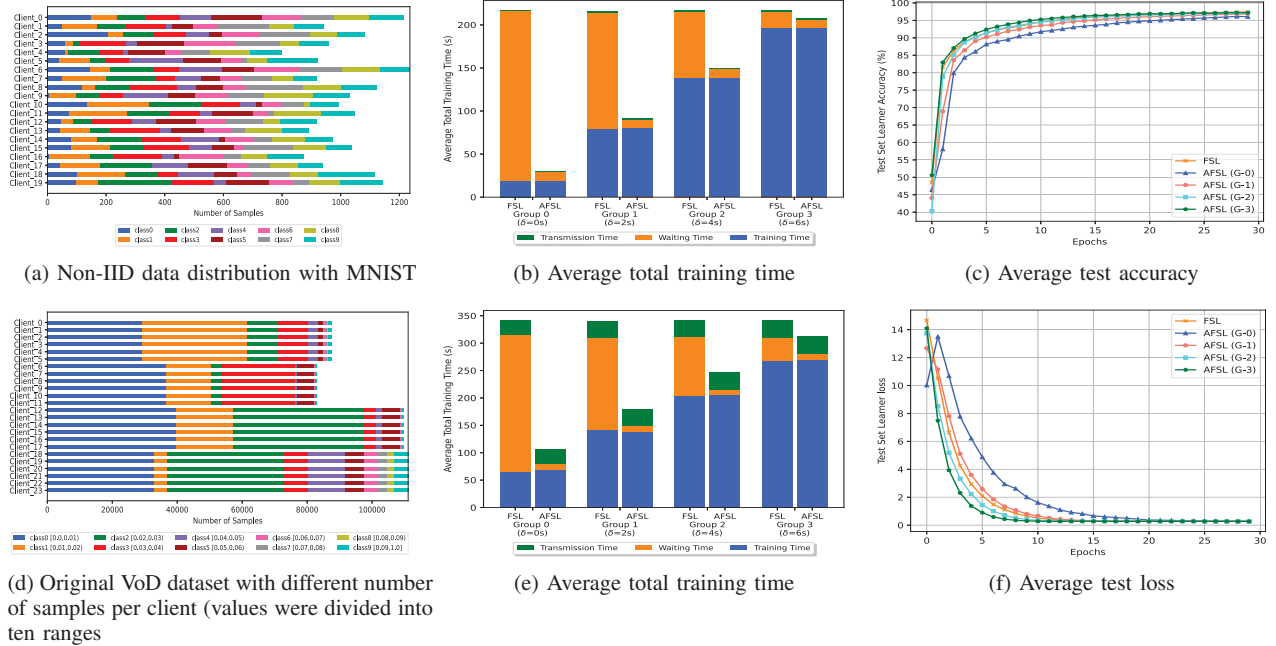


Fig. 4: Experiment II – Comparison between FSL & AFSL on MNIST (Figs. 4a, 4b and 4c) and on VoD (Figs. 4d, 4e and 4f) with heterogeneous clients. In this simulation, 20 clients were trained together but they were equally divided into four groups with different artificial delays. Non-iiid data was adopted.

all the clients begin the training at the same time. Moreover, to capture the deviation among different runs, all experiments were conducted five times, and results were averaged.

The time performance for each simulation is summarized in Fig. 3b and Fig. 3e for MNIST and VoD datasets, respectively. For both datasets, we can observe that the average training time per epoch is similar (i.e., time difference less than one second) for FLS and AFSL, with the time difference increasing slightly as the number of customers increases. Indeed, on the MNIST dataset, for 5, 20 and 40 clients, we observe that the average time necessary for the FSL to complete an epoch is increased by 0.31 s, 0.41 s and 0.54 s, respectively, compared to AFSL. This can be justified by the fact that the adopted multiprocessing scheme adds, by nature, some heterogeneity to the clients in terms of CPU capability. Therefore, in a scenario where the devices are homogeneous and the quality of connectivity to the server is identical, the devices end their epoch at approximately the same time. AFSL and FSL achieve similar learning times, with AFSL being slightly better than FSL, as it is difficult to achieve perfect homogeneity.

Regarding the accuracy of the models for the MNIST dataset and the loss (normalized mean absolute error) for the VoD dataset, respectively in Fig. 3c and Fig. 3f, both approaches showed a similar learning curve for all simulations. However, on the MNIST dataset, a slightly worse performance at the end of the training was noted for AFSL when compared to FSL. The observed difference in terms of accuracy for 5, 20, and 40 clients is, respectively, $\approx 0.7\%$, $\approx 0.5\%$ and $\approx 0.3\%$, which is quite negligible. Similarly, a negligible difference in loss is

observed on the VoD dataset. This means that in this scenario, AFSL performs as effectively as FSL. The similarity in performance can be attributed to the aggregation scheme, which prioritizes a more stable update of the global parameter server, reflecting the characteristics of the synchronous scheme.

B. Experiment II — FSL vs AFSL with heterogeneous clients

The second experiment aims to analyze the performance of AFSL and FSL in a scenario with heterogeneous devices. In other words, devices may have different compute capabilities and the available number of data samples for each user may be different. This is a very common scenario in the real world, and it's essential that AFSL works well in this case.

To emulate such a scenario using the MNIST dataset, we adopted a non-IID data distribution whereby the number of data points and class proportions is unbalanced. Samples are partitioned per user i following a normal distribution $|N(\mu, \sigma^2)|$ per each class. The μ value was empirically set to 100 and σ to 0.5μ . To give a better intuition, Fig. 4a illustrates the data distribution for a sample of 20 clients. As the VoD dataset is non-IID by nature, no modifications were required (as presented in Fig. 4d).

To simulate different compute capabilities, we introduced some artificial delay in an experiment with 20 clients on MNIST dataset and 24 clients on VoD dataset. Although all clients simultaneously train their own model in a cooperative way, we equally divided them into four different groups. Each group had a delay value based on a normal distribution $N(\mu_{delay}, \sigma_{delay}^2)$. Therefore, devices in the same group emu-

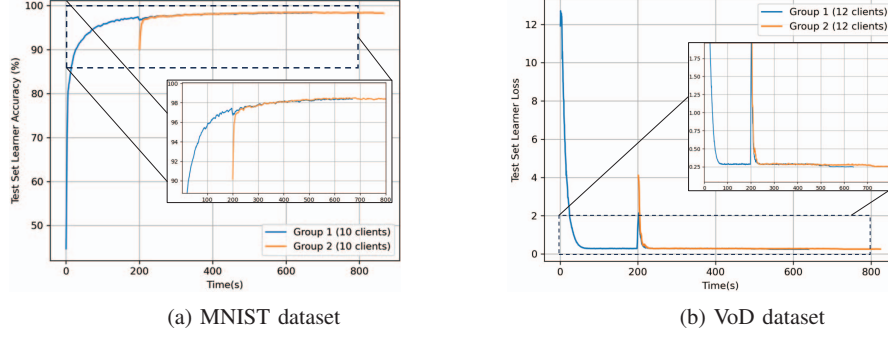


Fig. 5: Experiment III – AFSL performance analysis in a dynamic environment. In this experiment, two groups of clients starting the training in different times are trained together.

late devices with similar compute capabilities, represented by the same μ_{delay} , and difference in capability within a group is expressed by the deviation σ_{delay} . Considering as a baseline $2\times$ the average training time of Experiment I ($\approx 1s$ for 20 clients on MNIST), and to emulate clients with considerable less computational resources, the μ_{delay} values were set to 0s, 2s, 4s, 6s, respectively for the four groups of clients. The σ_{delay} value was set to $0.05\mu_{delay}$, to limit the range of variability within a group. The first group had $\mu_{delay} = 0s$, therefore no extra delay was added for the first group.

The results in terms of total training time per group are presented in Fig. 4b and Fig. 4e for MNIST and VoD dataset, respectively. When adopting AFSL on MNIST, a reduction in total training time of $\approx 86.5\%$, $\approx 57.5\%$, $\approx 31\%$ and $\approx 4.6\%$ was observed for groups 0, 1, 2 and 3, respectively. It can be observed that in a scenario with heterogeneous clients, the FSL approach has considerably poorer performance in terms of total time. Although the time spent on training (blue part) is similar in both approaches, in FSL the faster clients have to wait (orange part) for the slower ones to terminate their epoch in order to do the model aggregation and continue the training. On the other hand, in AFSL, each client can conclude the model aggregation step independently, right after they finish their epoch. Therefore, the average time to conclude the training is reduced, indicating that AFSL is better at exploiting all available resources. Similar results are obtained with the real-world VoD dataset, showing that the proposed solution is data agnostic. The communication time shown in green is much greater with the VoD dataset than with the MNIST dataset, due to the size of the smashed data being transferred in the network, which is higher with VoD than with MNIST.

In terms of accuracy on MNIST dataset, as presented in Fig. 4c, all groups reach 96% or more at the end of their training. In AFSL, groups 0, 1, 2 and 3, reach respectively, 96.05%, 96.84%, 97.1% and 97.2%, compared to 97.4% in the case of FSL. In AFSL, as the server model is updated as soon as the clients epochs are finished, the faster clients (i.e., clients of group 0, blue line) present the worst accuracy because they finish their epochs faster and, therefore, are exposed to less knowledge coming from the slower clients (via global

server S). On the other hand, slower clients present better accuracy because they take advantage of the multiple epochs performed by the faster clients. However, as the final model is defined based on the last version of the global server S, we can assume that AFSL showed a difference in accuracy of 0.2% when compared to FSL. On the VoD dataset, we observe a more stable loss at the end of training, which is similar for all the AFSL groups and with FSL. These results on the MNIST and VoD datasets demonstrate the effectiveness of AFSL, which brings a significant gain in terms of learning time compared with FSL, while maintaining good performance. The high-speed clients do not necessarily have to undergo additional training epochs. They can complete their training after completing the predetermined number of training epochs, ensuring that they obtain a good enough model. These clients will consequently utilize fewer resources, a vital characteristic for devices with limited computing capabilities, like IoT ones. It is noteworthy that with the asynchronous scheme, although it reduces the average training time, as well as the resource utilization, as observed, there may be a slight degradation in performance. Therefore, the selection of the aggregation principle helps mitigate the risk of accuracy degradation.

C. Experiment III — AFSL in a dynamic environment

In Experiment III, we study the performance of AFSL in a dynamic environment, where multiple clients with imbalanced data may connect/disconnect over time during the training phase. The dynamic nature is a common feature in various use cases for edge computing, spanning from smart factories and smart grids to autonomous vehicles and IoT applications. This experiment seeks to confirm the advantages of AFSL training in dynamic scenarios, i.e., provides robustness with a short recovery time by leveraging asynchronous client training. To replicate such a scenario, two sets of clients were simulated for each dataset. For MNIST, two groups of 10 clients each, and for VoD, two groups with 12 clients each. The first group begins training, while the second group joins the training after 200 seconds, once the learning curve of the first group stabilizes. The objective is to evaluate the impact of introducing new clients on the model's performance. To better observe the

impact of the client dynamics, the number of epochs for each group was set to 130. To represent unbalanced data, the non-IID data distribution presented in Experiment II was adopted (see Fig. 4a and Fig. 4d). The learning accuracy results are presented in Fig. 5. After joining the training, the second group takes advantage of the shared knowledge provided by the first group (via global server S) and learns faster.

For the MNIST dataset (Fig. 5a), the first group starts the training with an average test accuracy of 44.5% which eventually stabilizes near to 97.44% after ≈ 200 seconds. On the other hand, the second group starts the training with 90.1% accuracy which eventually reaches the precision of the first group after ≈ 10 seconds. It is worth mentioning that when the second group connects, a reduction in accuracy of $\approx 0.7\%$ is observed for the first group. This happens because the new group has not yet been through any back-propagation, and their gradients will behave more aggressively to reduce their loss, possibly affecting other clients. However, accuracy is recovered after a few seconds of training, when both groups begin to have similar average accuracy.

Similar results are observed on the VoD dataset, as shown in Fig. 5b, with the first group recovering its losses just seconds after the second group joined it. The proposed AFSL is therefore robust in a dynamic environment. Devices that join training late benefit from the knowledge acquired from other devices already participating in training, with a very short recovery time for the system as a whole. This is a crucial attribute because in a distributed learning system, new, cutting-edge devices join ongoing training. It is therefore critical that their addition does not significantly degrade overall performance, with the need for a very short recovery time.

IV. CONCLUSION

Recently, several studies have proposed combining FL and SL to take advantage of the merits of each approach and, at the same time, mitigate their drawbacks. However, research in this area is still in its early stages and asynchronous schemes for FL-SL combinations have not been investigated. As clients tend to be heterogeneous with respect to computation and communication environments as well as data distributions, asynchronous schemes play an important role in minimizing the effect of slower clients in the training process.

In this paper, unlike existing works, we design an asynchronous scheme for FL-SL (named AFSL) and investigate its performance by comparing it with one of the state-of-the-art frameworks in this domain, FSL. Results indicate that our approach can accelerate model learning by up to around 86% without sacrificing the model's convergence and accuracy. In addition, our approach has proven its robustness when clients have unbalanced data (non-IID) and when applied to dynamic environments where multiple clients can connect and disconnect over time during the training phase.

ACKNOWLEDGMENTS

First three authors were supported by a MITACS Globalink internship at Chalmers University of Technology, Sweden.

The work was also supported by the project "Multi-thread Federated Learning" sponsored by the CHAIR foundation at Chalmers University of Technology.

REFERENCES

- [1] V. Turina, Z. Zhang, F. Esposito, and I. Matta, "Federated or split? a performance and privacy analysis of hybrid split and federated learning architectures," in *IEEE 14th International Conference on Cloud Computing (CLOUD)*, 2021, pp. 250–260.
- [2] S. Li, L. Da Xu, and S. Zhao, "5G internet of things: A survey," *Journal of Industrial Information Integration*, vol. 10, pp. 1–9, 2018.
- [3] C. Stuart, Z. Filipovic, and J. Ramiro, "How is AI boosting network performance?" Sep 2023, Accessed: Sep. 12, 2023. [Online]. Available: <https://www.ericsson.com/en/blog/2023/9/ai-intelligent-automation-boosting-network-performance>
- [4] Q. Duan *et al.*, "Combined federated and split learning in edge computing for ubiquitous intelligence in internet of things: State-of-the-art and future directions," *Sensors*, vol. 22, no. 16, p. 5983, 2022.
- [5] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," *arXiv preprint arXiv:1610.02527*, 2016.
- [6] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.
- [7] J. Jeon and J. Kim, "Privacy-sensitive parallel split learning," in *International Conf. on Information Networking (ICOIN)*, 2020, pp. 7–9.
- [8] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8485–8493.
- [9] Y. Gao, M. Kim, C. Thapa, A. Abuadba, Z. Zhang, S. Camtepe, H. Kim, and S. Nepal, "Evaluation and optimization of distributed machine learning techniques for internet of things," *IEEE Transactions on Computers*, vol. 71, no. 10, pp. 2538–2552, 2021.
- [10] C. Xu, Y. Qu, Y. Xiang, and L. Gao, "Asynchronous federated learning on heterogeneous devices: A survey," *Computer Science Review*, vol. 50, p. 100595, Nov. 2023.
- [11] Z. Zhou, Y. Li, X. Ren, and S. Yang, "Towards efficient and stable k-asynchronous federated learning with unbounded stale gradients on non-IID data," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3291–3305, 2022.
- [12] Y. Chen, Y. Ning, M. Slawski, and H. Rangwala, "Asynchronous online federated learning for edge devices with non-IID data," in *IEEE International Conference on Big Data (Big Data)*, 2020, pp. 15–24.
- [13] X. Chen, J. Li, and C. Chakrabarti, "Communication and computation reduction for split learning using asynchronous training," in *IEEE Workshop on Signal Processing Systems (SiPS)*, 2021, pp. 76–81.
- [14] Z. Chai, Y. Chen, A. Anwar, L. Zhao, Y. Cheng, and H. Rangwala, "FedAT: A high-performance and communication-efficient federated learning system with asynchronous tiers," in *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021, pp. 1–16.
- [15] V. Stephanie, I. Khalil, and M. Atiquzzaman, "Digital twin enabled asynchronous SplitFed learning in E-healthcare systems," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 11, pp. 3650 – 3661, 2023.
- [16] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for federated learning on user-held data," in *NIPS Workshop on Private Multi-Party Machine Learning*, 2016, pp. 1 – 5.
- [17] Y. LeCun and C. Cortes, "MNIST handwritten digit database," 2010, Accessed: Jul. 30, 2023. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [18] Y. LeCun *et al.*, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [19] F. Samani, "Data traces for efficient learning on high-dimensional operational data," <https://github.com/foroughshh/KTH-traces>, accessed: Jul. 30, 2023.
- [20] R. Yanggratke *et al.*, "A service-agnostic method for predicting service metrics in real time," *International Journal of Network Management*, vol. 28, no. 2, p. e1991, 2018.